QI: Increase Your Quality Intelligence

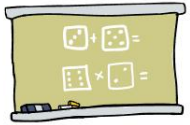
PNSQC.ORG

The Power of the Pause Button : The Superpower of Feature Flags and the Future of Software Delivery

Vaishnavi Venkata Subramanian

Hello,
I'm Vaishnavi

Things I ❤️‍🩹



Teaching

Public Speaking

Visual Communication

Things I do 🎭

Software Engineering

Ideation

Team collaboration

How to Give Me Feedback 👓

Direct constructive feedback provided with utmost honesty to help me grow.
Data driven and candid conversations with potential solutions for me to thrive and succeed.

Comms Preferences

Working Patterns 😊

- Morning Person
- Enjoy a quiet and vibrant environment
- Follow Pomodoro Technique 🍅

Things I struggle with 🥷

Mickey Mouse Job
#halfbaked

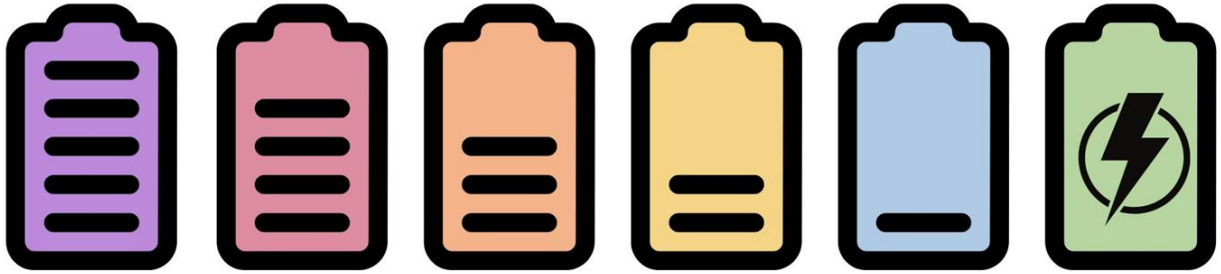
Yak Shaving

Interruptions

> "Simple things should be simple, complex things should be possible." - Alan Kay

P.S: No animals or disney characters were harmed in the making of this manual 🐁 🐭

April 2023
Version 3



Travel to event

Power packed Keynote

Tuning into plethora of sessions

Hours of being out

Network and socialize

Travel Home

Photo by Schiba on Unsplash

Martin Fowler
Chief Scientist,
Thoughtworks

Image : thoughtworks

Edith Harbaugh
CEO, Executive Chair
LaunchDarkly

Image : launchdarkly.com

What Are Feature Flags?

Software engineering technique that allows teams to enable or disable specific features in their applications dynamically, without having to redeploy the code.

Image : cloudbees

Feature Flags/Toggles

- Conditional switches in code to turn features ON or OFF at runtime.
- Like a “pause button” for any feature - no redeploy needed.
- Decouples the act of deploying code from the act of releasing features to users
- Teams can use feature flags to control exactly when and how users experience changes

Kill Switch for unexpected events

Image tenor

Why Use Feature Flags?

- Safe and gradual rollouts
- Provide a "kill switch" to instantly turn off problematic features without rolling back the entire deployment
- Increase development velocity
- Fast rollback

Why Use Feature Flags?

- Dark Launch: Deploy the new version of the application anytime. Since the new features are disabled, this doesn't immediately impact users.
- Enable features next day: Teams can turn on the feature flags based on business needs and conduct a gradual rollout.
- A/B Testing, Canary Release and Experimentation
- Improve collaboration - Decouple deployment from release

Experimentation through A/B Testing

Image : [scifi.net]

Types of Feature Toggles

- Release flags
- Experimentation flags
- Operational flags
- Permission flags

Release toggle

- Allows developers to merge non-ready code to production, turning it on once it’s ready, without the exposure to users.
- Who is responsible for toggling it: Developers.
- When should you delete it: A week or two after the release of the feature

Experimentation toggle

- Allows you to perform A/B testing. You divide the users into groups, each getting a different experience. While the toggle is active, you collect enough data to make the final decisions about which features to promote, refine, or retire.

- Who is responsible for toggling it: It varies, but mostly PMs.

- When should you delete it: a few days/weeks - once you get enough data to make a final decision.

Operational toggles

- May introduce when rolling out a new feature which has unclear performance implications so that system operators can disable or degrade that feature quickly in production if needed. Also called kill switches or circuit breakers

- Who is responsible for toggling it: Ops team - probably through a dedicated tool.

- When should you delete it: 1-2 weeks, once you gain confidence in the new feature. In rare cases, you might keep some “Kill Switches”, to stop non-critical functionality when the system is under unusually high load.

Permission toggles

- Allows one to change the features and experiences specific users have on the application. It may be through having “premium” features for paying customers, geography or “alpha” / “beta” features, for internal users / beta customers.

- This type is dangerous because it needs to be long-lived. Treating it the same way as the other toggles, as something that will be deleted soon, is a critical and common mistake.

- Who is responsible for toggling it: Set role-based access control (RBAC) to specify who can delete flags in a given environment.

- When should you delete it: Collective decision. Enable monitoring and observability. Thorough review and documentation

Product Adoption Curve

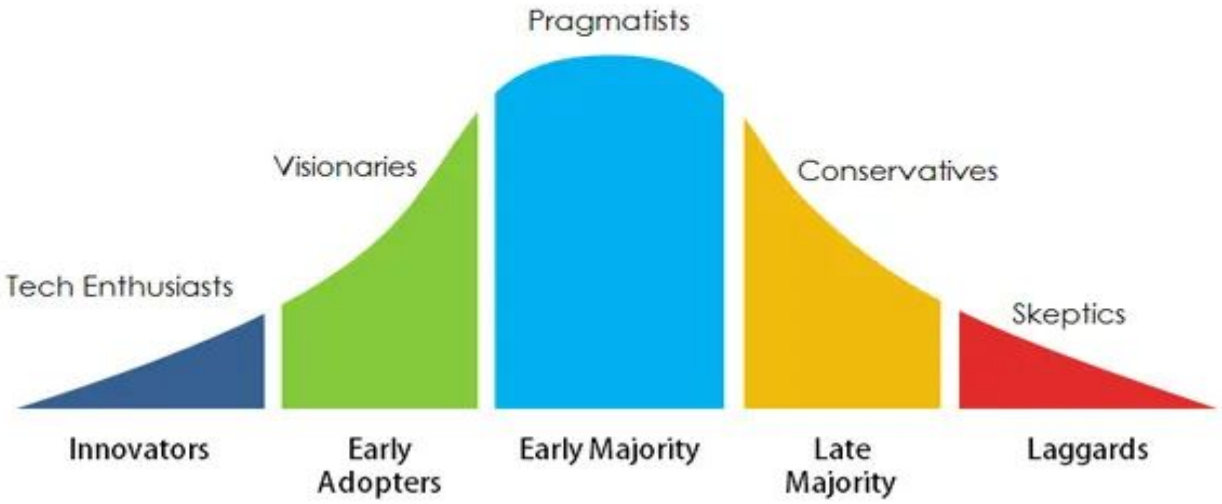


Image : prodpad.com

The double edged sword!

Knight Capital Group

Power Peg and SMARS (Smart Market Access Routing System)

August 1st, 2012, preparing for NYSE’s Retail Liquidity Program

Power Peg and SMARS (Smart Market Access Routing System)

- Power Peg - Trading algorithm that bought high and sold low; it was specifically designed to move stock prices higher and lower in order to verify behavior of its other proprietary trading algorithms in a controlled environment. Last used in 2003
- It was not to be used in the live, production environment.
- The errant code created a "ping-ponging" pattern of buying at the ask price (higher) and selling at the bid price (lower)—the exact opposite of profitable trading. Some stocks experienced dramatic price movements

How to lose half a billion dollars with incorrect feature flags and a deployment failure

Image : [statsig.com]

“Murphy's Law applies double in production environments”

Credit : soundofdevelopment

Dead Code is Dead Weight : Unused code shouldn't just lie dormant in your production systems but must be removed entirely.

Mistakes to Avoid when Adopting Feature Flags

- Leaving stale flags in code causing a debt
- Never reuse a toggle
- Poor flag naming conventions
- Lack of documentation
- Performance bottlenecks
- Security and Access Control

Image : flickr

Good names

- Are descriptive. For example, is-v2-billing-dashboard-enabled is much clearer than is-dashboard-enabled. It includes useful version and product context.
- Use name "types". This helps organize them and makes their purpose clear. Types might include experiments, releases, and permissions. For example, instead of new-billing, they would be new-billing-experiment or new-billing-release.
- Reflect their return type. For example, is-premium-user for a boolean, enabled-integrations for an array, or selected-theme for a single string.
- Use positive language for boolean flags. For example, is-premium-user instead of is-not-premium-user. This helps avoid confusing double negatives when checking the flag value (e.g. if !is_not_premium_user).

Source : posthog.com

The Meta Game Problem

- Manual cleanup doesn’t work - devs are busy
- Flags become permanent, codebase gets messy
- Bugs and confusion increase over time

What’s wrong with the feature flags addiction

Feature Flags have a tendency to multiply rapidly, particularly when first introduced.

They are useful and cheap to create and so often a lot are created.

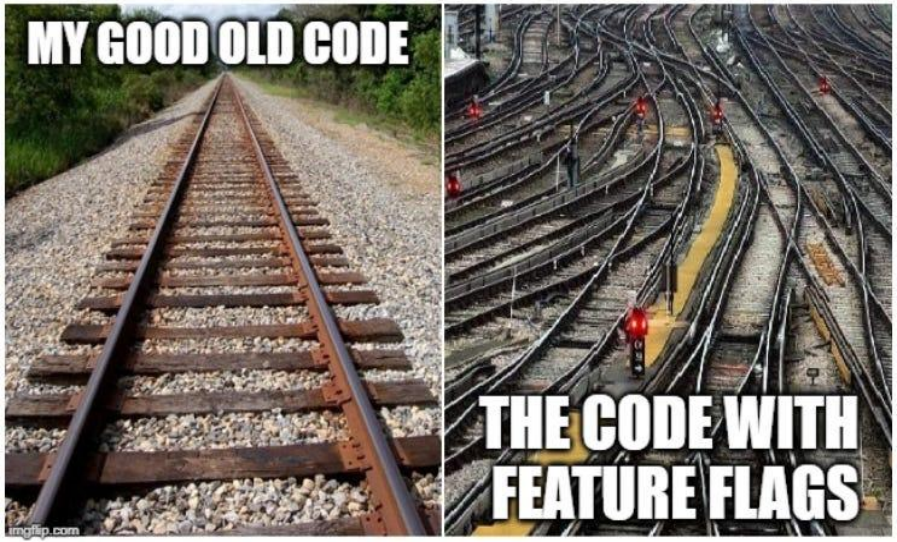


Image manager.dev

Conflicting Flags

Overused Flags

Image : reddit

Best Practices for Scaling Feature Flag Management

- Centralized Flag Management Tools
- Flag carefully. Define Flag Lifecycles
- Integrate with CI/CD Pipelines
- Observability and Monitoring
- Team Education
- Trunk Based Delivery

Flag Lifecycle

- Have a designated owner responsible for reviewing and monitoring flag’s status
- Remove it when it is no longer needed
- Regular audits of the codebase
- Automated tools to track flag usage
- Clear documentation can all help ensure that stale flags are identified and eliminated promptly
- No expiry? Build fails.
- Automated detection and alerts. When a flag hits the criteria, alert someone (like the owner) that it might be time to remove it. Keep reminding them until they do.

AND THIS IS A ROOM WHERE WE KEEP OUR DOCUMENTATION

Dataedo /cartoon

Piotr@Dataedo

Documentation

- Flag’s purpose
- Owner
- Creation date
- Always provide default behavior for each flag if the configuration system fails or the flag value is undefined.
- Planned removal date

Observability and Monitoring

Make Alerts Matter

You cannot resolve a broken feature without knowing it's broken

Akin to having a sprinkler system without a smoke detector

The “Pause Button” Metaphor

- Instantly enable or disable features
- Reduce risk during launches
- Respond to issues in real-time

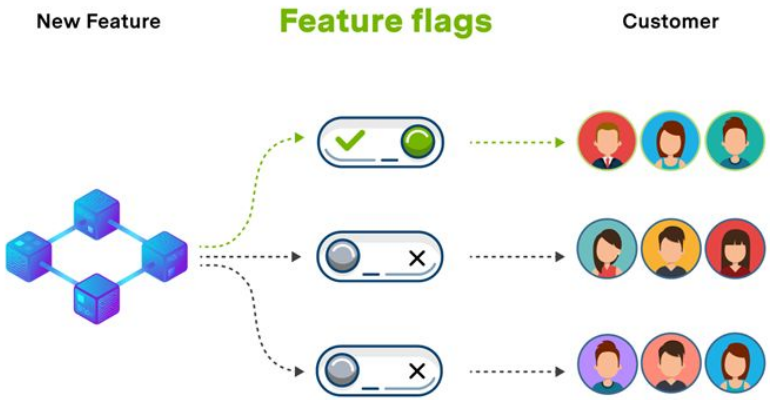
Feature Flag Decoupling

- Release code with features OFF, enable later
- No redeploy needed, everything works

Superpowers Unlocked

- Safe Deployments: Ship code with features OFF, enable later
- Instant Rollbacks: Turn off features if issues arise. No redeployment
- Progressive Delivery: Canary, beta, or region-based rollouts

Real-World Example



Feature Flags Customer Lifecycle

Image : https://medium.com/@ridvanozcan48

Verbose Logging for Troubleshooting

Activate various levels to reproduce

https://medium.com/shipbook/android-log-levels-c0313055fdb9

Real-World Example

- New UI shipped to production, but only 5% of users see it
- If bugs appear, instantly revert for all users—no redeploy

Google Cloud Suffers Major Disruption

In June 2025 Google Cloud, Google Workspace and Google Security Operations products experienced increased 503 errors in external API requests, impacting customers.

According to the incident report by Google, on May 29, 2025, a new feature was added to Service Control for additional quota policy checks. This code change and binary release went through their region by region rollout, but the code path that failed was never exercised during this rollout due to needing a policy change that would trigger the code. As a safety precaution, this code change came with a red-button to turn off that particular policy serving path.

The issue with this change was that it did not have appropriate error handling nor was it feature flag protected.

Pitfalls

- Too many flags (“flag debt”)
- Flags never cleaned up
- Hard-to-test flags (e.g., hidden in databases)
- Configuration drift between environments - Development and Production flags are different
- New developers start, current flag configuration is unknown until runtime

The Solution: Mandatory Expiry

- Every flag must have:
  - An expiry date (hard-coded)
  - A permanent value after expiry (ON/OFF/variant)
- No expiry? Build fails.

Enforced by Code

- After expiry, flag always returns the permanent value
- No manual intervention needed
- No more flag debt

Bad Practise - Triangle of Doom

```php
<?php

// Bad

$cats = [];

foreach ($taxonomy['Kingdoms'] as $kingdom) {

    if ($kingdom['Name'] === 'Animalia') {

        foreach ($kingdom['Phyla'] as $phylum) {

            if ($phylum['Name'] === 'Chordata') {

                foreach ($phylum['Classes'] as $class) {

                    if ($class['Name'] === 'Mammalia') {

                        foreach ($class['Orders'] as $order) {

                            if ($order['Name'] === 'Carnivora') {

                                foreach ($order['Families'] as $family) {

                                    if ($family['Name'] === 'Felidae') {

                                        foreach ($family['Genera'] as $genus) {

                                            if ($genus['Name'] === 'Felis') {

                                                foreach ($genus['Species'] as $species) {

                                                    if ($species['Name'] === 'F. catus') {

                                                        // Found a cat! Add it to our $cats array
                                                        $cats[] = $animal;

                                                    }

                                                }

                                            }

                                        }

                                    }

                                }

                            }

                        }

                    }

                }

            }

        }

    }

}
```

Good Practise - Dependency Injection

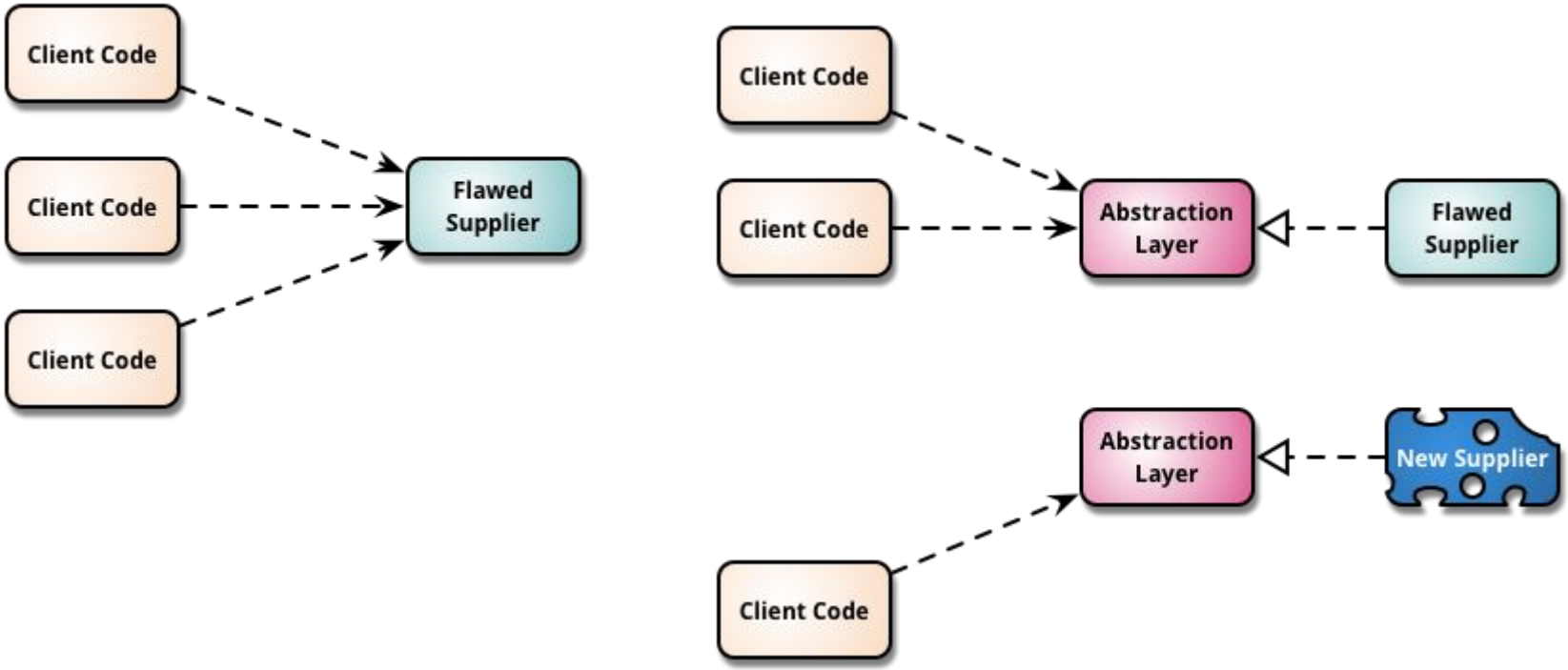
```
interface IFeature
{
  void Execute();
}

class FeatureA : IFeature
{
  public void Execute(){}
}

class FeatureB : IFeature
{
  public void Execute(){}
}

// use dependency injection to resolve the object that meet the condition
IFeature feature = isFeatureA ? new FeatureA() : new FeatureB();
      feature.Execute();
```

Branch By Abstraction



Branch By Abstraction

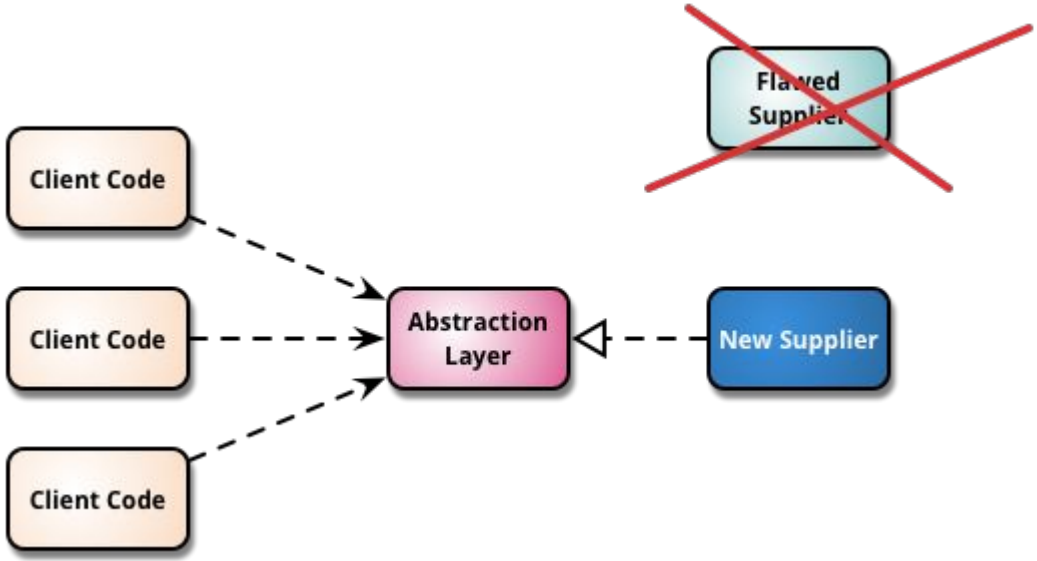
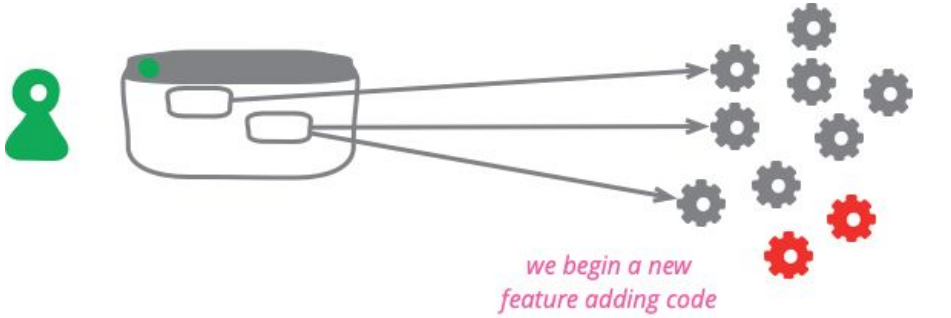
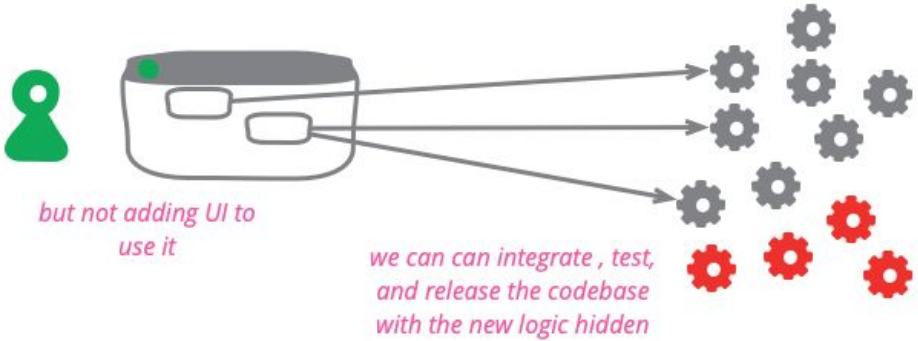
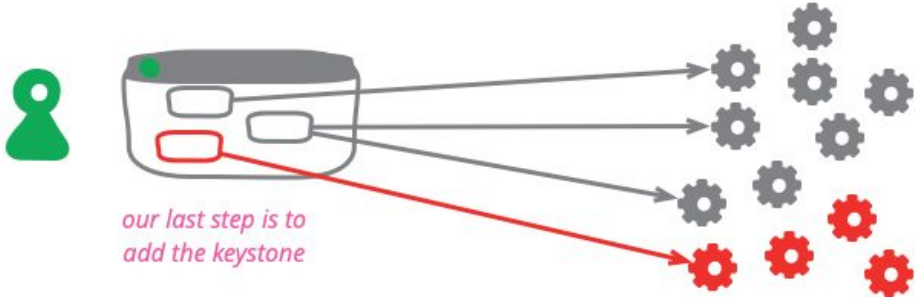


Image : [martinfowler.com]







Keystone Interface

Image : [martinfowler]

Key Takeaways

- Feature flags is a software superpower
- “Pause button” for risk-free delivery
- Mandatory expiry & permanent value = no flag debt
- Enforced by code, not memory.
- Don’t be afraid to sunset features
- Branch by Abstraction

The Future of Software Delivery

- Feature flags as first-class citizens
- Automated, disciplined flag lifecycle
- Safer, faster, and smarter releases

Software Practices

Solving the Woes - Stale Flags & Technical Debt

Performance

Ownership & Documentation

Security

Collaboration

Experimentation

Avoid Triangle of Doom - multiple if-else


DO NOT
PUSH
BUTTON


DO
PUSH
BUTTON

“If you love something, set it free.”
The same is true of feature flags

Thank you!

Connect with me!